Paradigmas de Programação

Programação Funcional: Lista 1

Parte A - Funções

1. Usando apenas as funções uma ou mais dentre as funções `head`, `tail`, `take`, `drop`, `length` e `null`, e os recursos e elementos presentes nas duas primeiras aulas sobre programação funcional, implemente as funções abaixo:

(a) `second :: [a] -> Maybe a`, que recebe uma lista como parâmetro e retorna o segundo elemento da lista, se existir. Ex.:

```
ghci> second [1, 2, 3, 4, 5]        -- retorna Just 2
ghci> second [1]                    -- retorna "Nothing"
```

Solução:

O padrão `(_ : x : _)` só casa com listas de pelo menos dois elementos e captura o segundo em `x`; qualquer outra lista (vazia ou unitária) cai no segundo caso.

```haskell
-- 1.a second
second :: [a] -> Maybe a
second (_ : x : _) = Just x
second _           = Nothing
```

Solução alternativa: em vez de casar o padrão, testa-se o tamanho com `length`; se há pelo menos dois elementos, o segundo é o primeiro da cauda (`head (tail xs)`).

```haskell
second_alt1 :: [a] -> Maybe a
second_alt1 xs
    | length xs < 2 = Nothing
    | otherwise     = Just $ head $ tail xs
```

(b) `body :: [a] -> Maybe [a]`, que recebe uma lista como parâmetros e retorna todos os elementos, exceto o primeiro e o último, se possível. Ex.:

```
ghci> body [1, 2, 3, 4, 5]          -- retorna Just [2, 3, 4]
ghci> body [1]                      -- retorna "Nothing"
```

Solução:

Listas vazias e unitárias não têm primeiro e último elementos distintos, então o resultado é `Nothing`. Nos demais casos, o padrão `(_:xs)` descarta o primeiro elemento, e `take (length xs - 1) xs` descarta o último. Para uma lista de dois elementos, o resultado é `Just []`.

```haskell
-- 1.b body
body :: [a] -> Maybe [a]
body []     = Nothing
body [_]    = Nothing
body (_:xs) = Just $ take (length xs - 1) xs
```

Solução alternativa: com menos de dois elementos não há o que devolver. Caso contrário, `tail` descarta o primeiro elemento e `take (length xs - 2)` guarda os $N-2$ elementos seguintes, que são todos, exceto o primeiro e o último.

```haskell
body_alt1 :: [a] -> Maybe [a]
body_alt1 xs
    | length xs < 2 = Nothing
    | otherwise     = Just $ take (length xs - 2) $ tail xs
```

(c) `median :: [a] -> Maybe a`, que retorna o elemento central da lista de N elementos, se N é ímpar, ou o $N/2$-ésimo elemento, se N é par, se possível. Ex.:

```
ghci> median [1, 2, 3, 4, 5, 6]      -- retorna Just 3
```

Solução:

Em ambos os casos o elemento procurado é o de posição $\lfloor (N+1)/2 \rfloor$ (contando a partir de 1): se N é ímpar, é o elemento central; se N é par, é o $N/2$-ésimo. Como `drop` descarta os k primeiros elementos, descartamos $\lfloor (N+1)/2 \rfloor - 1$ elementos e pegamos o `head` do resto.

```haskell
-- 1.c median
median :: [a] -> Maybe a
median [] = Nothing
median xs = Just $ head $ drop ((div (length xs + 1) 2) - 1) xs
```

Solução alternativa: `div (length xs - 1) 2` é o número de elementos a descartar (igual a $\lfloor (N+1)/2 \rfloor - 1$), o que evita o "mais um, menos um".

```haskell
median_alt1 :: [a] -> Maybe a
median_alt1 xs
    | null xs   = Nothing
    | otherwise = Just $ head $ drop (div (length xs - 1) 2) xs
```

2. Utilizando **obrigatoriamente** recursão, e opcionalmente guardas e os recursos apresentados nas duas primeiras aulas sobre programação funcional, implemente as funções abaixo:

(a) `parity :: Int -> Int`, que recebe um inteiro positivo n e retorna 0 se n é par, ou 1, se n é ímpar.

Solução:

Subtraindo 2 de n não se altera a paridade; a recursão termina em 0 (par) ou 1 (ímpar), que são exatamente os valores a retornar.

```haskell
-- 2.a parity
parity :: Int -> Int
parity n
    | n <= 1    = n
    | otherwise = parity $ n - 2
```

Solução alternativa: a paridade de n é o oposto da paridade de $n-1$;
logo, `parity n = 1 - parity (n - 1)`, com `parity 0 = 0`.

```haskell
parity_alt1 :: Int -> Int
parity_alt1 n
    | n == 0    = 0
    | otherwise = 1 - parity_alt1 (n - 1)
```

(b) `remainder :: Int -> Int -> Int`, que recebe dois inteiros positivos a e b e retorna o resto da divisão euclidiana de a por b.

Solução:

Se $a < b$, o resto é o próprio a; caso contrário, o resto de a por b é igual ao resto de $a - b$ por b.

```haskell
-- 2.b remainder
remainder :: Int -> Int -> Int
remainder a b
    | a < b     = a
    | otherwise = remainder (a - b) b
```

(c) `quotient :: Int -> Int -> Int`, que recebe dois inteiros positivos a e b e retorna o quociente da divisão euclidiana de a por b.

Solução:

Se $a < b$, o quociente é 0; caso contrário, é uma unidade a mais que o quociente de $a - b$ por b (cabe mais um b em a).

```haskell
-- 2.c quotient
quotient :: Int -> Int -> Int
quotient a b
    | a < b     = 0
    | otherwise = 1 + quotient (a - b) b
```

(d) `greater_common_divisor :: Int -> Int -> Int`, que recebe dois inteiros positivos a e b e retorna o maior divisor comum (a, b) de a e b. Dica: $(a, b) = (b, a - b)$ e $(a, 0) = a$.

Solução:

Caso base: $(a, 0) = a$. Passo recursivo: o algoritmo de Euclides, $(a, b) = (b, a \bmod b)$, que é a dica $(a, b) = (b, a - b)$ aplicada repetidamente até que $a < b$. O resto é calculado pela função `remainder` do item anterior.

```haskell
-- 2.d greater_common_divisor
greater_common_divisor :: Int -> Int -> Int
greater_common_divisor a b
    | b == 0    = a
    | otherwise = greater_common_divisor b $ remainder a b
```

Solução alternativa: só pela dica do enunciado, usando apenas subtração: se $a \geq b$, então $(a, b) = (b, a - b)$; se $a < b$, só trocar a ordem dos argumentos.

```haskell
greater_common_divisor_alt1 :: Int -> Int -> Int
greater_common_divisor_alt1 a b
    | b == 0    = a
    | a >= b    = greater_common_divisor_alt1 b (a - b)
    | otherwise = greater_common_divisor_alt1 b a
```

(e) `semifactorial :: Int -> Int`, que recebe um inteiror positivo e retorna

$$\texttt{semifactorial}\ n = n(n-2)(n-4)\ldots 1$$

e `semifactorial` $1 =$ `semifactorial` $0 = 1$.

Solução:

Os casos base são $n = 0$ e $n = 1$ (o produto é 1); para $n \geq 2$, $n!! = n \times (n-2)!!$.

```haskell
-- 2.e semifactorial
semifactorial :: Int -> Int
semifactorial n
    | n <= 1    = 1
    | otherwise = n * (semifactorial $ n - 2)
```

(f) `power :: Int -> Int -> Int`, que recebe dois inteiros positivos a e n e retorna a^n.

Solução:

$a^0 = 1$ e $a^n = a \times a^{n-1}$.

```haskell
-- 2.f power
power :: Int -> Int -> Int
power a n
    | n == 0    = 1
    | otherwise = a * (power a $ n - 1)
```

(g) `sum_of_first_n :: Int -> Int`, que recebe um natural n, e retorna a soma dos primeiros n números naturais.

Solução:

A soma de zero termos é 0, e $\sum_{i=1}^{n} i = n + \sum_{i=1}^{n-1} i$.

```haskell
-- 2.g sum_of_first_n
sum_of_first_n :: Int -> Int
sum_of_first_n n
    | n == 0    = 0
    | otherwise = n + (sum_of_first_n $ n - 1)
```

(h) `sum_of_first_n_squares :: Int -> Int`, que recebe um natural n, e retorna a soma dos quadrados dos primeiros n números naturais.

Solução:

Análogo ao item anterior: $\sum_{i=1}^{n} i^2 = n^2 + \sum_{i=1}^{n-1} i^2$.

```haskell
-- 2.h sum_of_first_n_squares
sum_of_first_n_squares :: Int -> Int
sum_of_first_n_squares n
    | n == 0    = 0
    | otherwise = (n * n) + (sum_of_first_n_squares $ n - 1)
```

(i) `arithmetic_progression_sum :: Int -> Int -> Int -> Int`, que recebe três naturais a, r e n e retorna a soma dos primeiros n termos de uma progressão aritmética cujo primeiro termo é igual a a e a razão é igual a r.

Solução:

A soma dos n termos é o primeiro termo a mais a soma dos $n-1$ termos da progressão que começa em $a+r$ com a mesma razão.

```haskell
-- 2.i arithmetic_progression_sum
arithmetic_progression_sum :: Int -> Int -> Int -> Int
arithmetic_progression_sum a r n
    | n == 0    = 0
    | otherwise = a + arithmetic_progression_sum (a + r) r (n - 1)
```

Solução alternativa: soma-se primeiro o *último* termo, $a+(n-1)r$, e volta para $n-1$ termos; assim, a e r estão fixos em toda a recursão.

```haskell
arithmetic_progression_sum_alt1 :: Int -> Int -> Int -> Int
arithmetic_progression_sum_alt1 a r n
    | n == 0    = 0
    | otherwise = (a + (n - 1) * r) + arithmetic_progression_sum_alt1 a r (n - 1)
```

(j) `geometric_progression_sum:: Int -> Int -> Int -> Int`, que recebe três naturais a, q e n e retorna a soma dos primeiros n termos de uma progressão geométrica cujo primeiro termo é igual a a e a razão é igual a q.

Solução:

Como $a+aq+aq^2+\cdots+aq^{n-1} = a+q\,(a+aq+\cdots+aq^{n-2})$, a soma de n termos é a mais q vezes a soma de $n-1$ termos da mesma progressão.

```haskell
-- 2.j geometric_progression_sum
geometric_progression_sum :: Int -> Int -> Int -> Int
geometric_progression_sum a q n
    | n == 0    = 0
    | otherwise = a + q * geometric_progression_sum a q (n - 1)
```

Solução alternativa: soma-se o último termo, $a\,q^{n-1}$, e volta para $n-1$ termos.

```haskell
geometric_progression_sum_alt1 :: Int -> Int -> Int -> Int
geometric_progression_sum_alt1 a q n
    | n == 0    = 0
    | otherwise = a * q ^ (n - 1) + geometric_progression_sum_alt1 a q (n - 1)
```

(k) `binomial :: Int -> Int -> Int`, que recebe dois inteiros não-negativos n e m, com $n \geq m$, e computa o coeficiente binomial $\binom{n}{m}$.

Solução:

Usamos a relação de Pascal, $\binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1}$, com casos base $\binom{n}{n} = \binom{n}{0} = 1$. A primeira guarda ($m > n$, que retorna 0) não é alcançada para entradas válidas; serve apenas como proteção.

```haskell
-- 2.k binomial
binomial :: Int -> Int -> Int
binomial n m
    | m > n                 = 0
    | (n == m) || (m == 0) = 1
    | otherwise             = binomial (n - 1) m + binomial (n - 1) (m - 1)
```

Solução alternativa: usa a identidade $\binom{n}{m} = \frac{n}{m}\binom{n-1}{m-1}$ se e apenas se $m \leq n$, que reduz n e m ao mesmo tempo e faz apenas uma chamada recursiva.

```haskell
binomial_alt1 :: Int -> Int -> Int
binomial_alt1 n m
    | m == 0    = 1
    | otherwise = binomial_alt1 (n - 1) (m - 1) * n `div` m
```

(l) `count_vowels :: String -> Int`, que recebe uma string de caracteres minúsculos s e retorna o número de vogais ("$aeiou$") presentes em s.

Solução:

Uma string vazia tem 0 vogais. Caso contrário, somamos 1 se o primeiro caractere é uma vogal e continuamos a contagem no restante da string.

```haskell
-- 2.l count_vowels
count_vowels :: String -> Int
count_vowels [] = 0
count_vowels (c:cs)
    | c `elem` "aeiou" = 1 + count_vowels cs
    | otherwise        = count_vowels cs
```